

Autonomous armor for software.

An authorised, defensive Cyber-Reasoning System for AI Kavach. Given a C/C++ repository you own, Brahmadata finds a real memory-safety defect with its own fuzzing, drafts a patch with a self-hosted model, and lets deterministic tools – not the model – deliver the verdict.

[PROBLEM]

Scanners and “AI security” tools stop at the report. They hand a human a finding and a suggested patch – and a suggested patch, accepted on model confidence, is exactly how a plausible-looking wrong fix ships to production.

[THE DIFFERENTIATOR]

The same pipeline visibly **rejects** a patch that eliminates the crash but breaks the program's behaviour. Every competitor shows a patch that worked; we show our system refusing one that only looked like it did.

[IDEA & MOTIVATION]

Close the loop, and make the machine check its own work. Discover → minimise to a deterministic reproducer → patch with a local model → let the reproducer, the regression suite, static checks and renewed fuzzing decide.

[DEFENSIVE / AUTHORISED-USE BOUNDARY]

Authorised repositories and isolated environments only; write-once authorization record + server-verified snapshot hash. No public-target scanning, no exploit deployment, no automatic production merge. Repository content never leaves for a hosted inference API – structurally enforced.

One pipeline. Unattended after a single [START].

[TIER 1 · DETERMINISTIC TRIAGE]

- **01 Authorize + snapshot.** Immutable snapshot; sha256 recomputed server-side and matched.
- **02 Baseline.** configure → build → CTest on the pristine tree. Pass/fail counts recorded – the denominator every later verdict is checked against.
- **03 Analyze.** Semgrep (offline C/C++ ruleset) + compiler warnings parsed into structured findings; git-history summary.

[TIER 2 · SANDBOX + LIGHTWEIGHT PATCH]

- **04 Stress-test.** ASan/UBSan build + libFuzzer campaign in a locked-down container. Sanitizer-confirmed heap-buffer-overflow with a stack trace; crashes deduplicated.
- **05 Correlate + minimise.** Bind the crash to a source location; minimise the input to a reproducer that replays 5/5 from a clean build; persist it durably.
- **06 Patch.** A self-hosted CodeLlama 7B model gets the crash report + localised source. Patch policy (single file, allowlist, changed-line cap) must pass **before** compile.

[VERIFICATION & PROOF-OF-FIX]

- **07 Verify** – every policy-passing candidate, identical gate sequence. The verifier is provenance-blind: it cannot see the model, its confidence, or its rationale.
- **08 Proof-of-fix.** The verdict is derived from the gate matrix, never from confidence.
- **09 Export + teardown.** report.md / .json, gate matrix, content-addressed artifacts, sha256 manifest. Sandbox + model-host release confirmed, zero strays.

[STEP 08 · TWO CANDIDATES, ONE GATE MATRIX, ONE RUN]

+ COMPILE	cmake · exit 0
+ REPRODUCER_ELIMINATED	replay clean
+ REGRESSION_PRESERVED · patch A	8/8 ctest
× REGRESSION_PRESERVED · patch B	1 of 8 failed
– STATIC_DELTA · RENEWED_FUZZING	disclosed, with reason

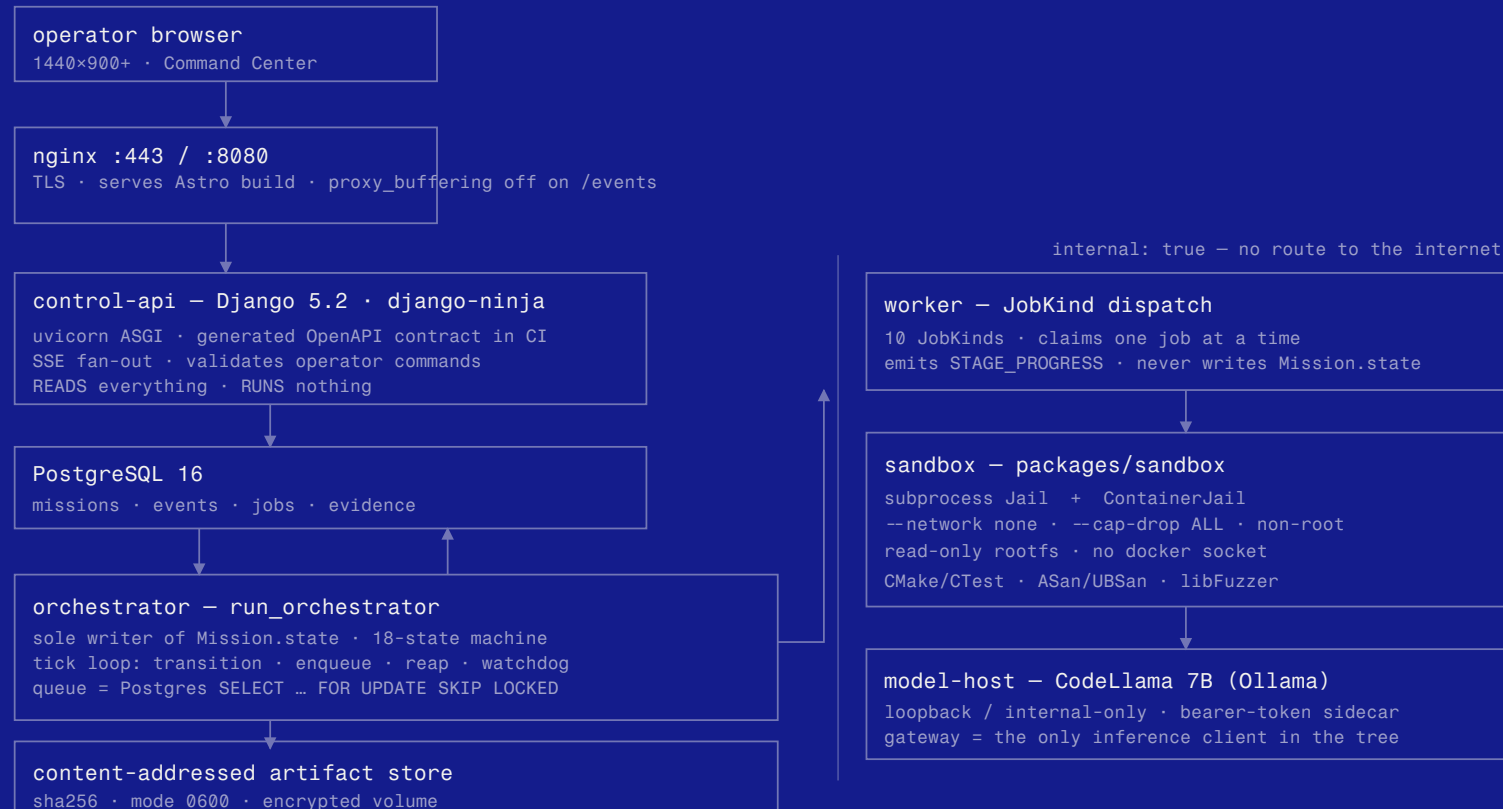
[NOTHING ESCALATES A TIER ON A GUESS]

[WHAT THE LOOP CATCHES]

Patch A – the correct fix – is marked **VERIFIED**. Patch B kills the crash by deleting the feature that was overflowing: it passes compile, eliminates the reproducer, then **fails a regression test** (six assertions in the tab-expansion case) → **REJECTED**, beside the accepted one. An overfit patch that fools every static gate is caught by renewed fuzzing of the patched build.

[WORK MOVES UP ONLY WHEN THE EVIDENCE IN HAND JUSTIFIES THE COST]

Four processes. No broker. No hosted API.



Fifteen deployable units in the original design collapse to four application processes. The worker is the only thing that holds repository content, and the only caller of the model.

[STACK — IMPLEMENTED]

- + Astro 7 + React islands · SVG Core
- + Django · django-ninja · Pydantic 2
- + PostgreSQL 16 · ORM + migrations
- + 18-state machine · DB job queue
- + Semgrep · compiler-warning parsing
- + libFuzzer · ASan/UBSan · crash dedup
- + rootless Docker · Jail + ContainerJail
- + self-hosted CodeLlama 7B · loopback
- + 5-gate matrix · no confidence arg

[FINALE / ROADMAP]

- > Live-model patch inside the full run
- > Semgrep + renewed-fuzz in a live mission
- > ContainerJail default everywhere
- > Tier 3 — designed path only, unnamed

The system rejects its own patch — the rejection is a real regression-test failure, not a scripted demo beat.

[NOVELTY — VERIFICATION THAT OVERRULES THE MODEL]

- **Evidence-gated escalation.** Nothing reaches the model until deterministic tiers produce a confirmed, minimised finding.
- **No confidence path.** `derive_verdict()` takes one argument — the gate matrix. A VERIFIED record over a failing gate cannot be constructed.
- **Provenance-blind verifier.** The operator-supplied “bad” candidate takes a genuinely identical path.
- **Overfit caught by renewed fuzzing** of the patched build — not by a human already knowing which input to try.
- **Disclosure as a feature.** A gate that did not run is as loud as a failure; the verdict carries its denominator.

[POSTURE — SELF-HOSTED · AUTHORISED · ISOLATED]

- **Repository content never reaches a hosted API.** Enforced three ways: internal-only compose network, a single-inference-client source test, a boot-time system check that refuses a hosted endpoint.
- **Rootless isolated execution** — `--network none`, all capabilities dropped, non-root, read-only rootfs. Egress denial proven by a live DNS+TCP test.
- **Authorised repositories only** — write-once record, server-verified snapshot hash. No public scanning, no exploit deployment, no auto-merge.
- **Operator-visible Command Center** — every panel reads the same stream that produces the evidence bundle.

[RESOURCE UTILISATION & LIGHTWEIGHT DESIGN]

- **CPU-first.** Rented GPU cut entirely. A local 7B model with a process-level lease and confirmed teardown.
- **4 application processes** vs 15 in the original design. Postgres SKIP LOCKED instead of a message broker. No Redis, no S3, offline Semgrep ruleset.
- **Runs on one machine, offline.** Orchestrator / worker / gateway / evidence are modules in one Django project.

~48s

FULL PIPELINE,
UNATTENDED

0.3s

FUZZ TO SEEDDED
DEFECT

Demonstrated today. Sharpened at the finale.

[WHAT THE PROTOTYPE DEMONSTRABLY DOES — LIVE, UNATTENDED, REAL HTTP API]

- Full nine-step pipeline end-to-end in 47.75 s wall-clock.
- A **self-discovered**, ASan-confirmed heap-buffer-overflow in an authorised C target, with a durable, deterministic reproducer.
- Two verdicts from one mission — one patch → **VERIFIED** (first end-to-end in project history); one plausible crash-only patch → **REJECTED** on a real regression failure, same gate matrix.
- An **exported evidence bundle** — snapshot hash, crash report, minimised input, both diffs, both gate matrices, both verdicts — sha256-manifested and **independently read back** by someone who did not build the system.
- Confirmed teardown, zero stray containers. Self-hosted CodeLlama 7B: **10 / 10** generation attempts returned schema- and policy-valid candidates.
- Command Center rebuilt to the approved spec; **≈900+** automated tests; CI gated on a real PostgreSQL; egress denial proven live.

[EXPECTED COMPETITION OUTPUT]

A reproducible C/C++ pipeline that, on an authorised repository, produces a confirmed finding, a minimal patch, a deterministic verdict, and a portable evidence bundle a judge can audit offline. Shortlist-ready now: the loop, the rejection, the disclosure discipline, the self-hosted / authorised posture, the footprint.

[REFINED DURING THE 36-HOUR GRAND FINALE]

- Live-model patch generation inside the full unattended run (finale hardware clears the dev-VM memory ceiling).
- Semgrep + renewed-fuzzing gates exercised inside a full live mission.
- ContainerJail as the default isolation for every stage, every deployment.
- Three timed rehearsals with failure injection: GPU-unavailable, target won't build, stage hangs.
- A real open-source C target with a known historical CVE, alongside the purpose-built one. Human-recorded fallback demonstration.

PERFORMANCE ~48 s pipeline · **SPEED** sub-second reproducer replay · **PRECISION** deterministic gates + demonstrated rejection · **FUNCTIONALITY** nine-step loop + live Command Center · **SCALABILITY** stateless processes, SKIP LOCKED queue, adapter generalised beyond the demo target